WEB ENGINEERING SEMESTER PROJECT

AuraBeat

AI-Powered Music Generation Platform

Final Project Lab Report

Course	Web Engineering
Assessment	Final Project Lab Submission
Project Domain	AI-powered music generation web application
Technology Stack	Next.js 14, TypeScript, Tailwind CSS, Supabase, Zustand, Meta MusicGen-small, Google Colab Gradio, Vercel
Prepared By	ABDUL WAHAB (007), TALHA NAVEED (089)
Date	May 18, 2026

REPORT BASIS: This report is prepared as the final lab submission for the AuraBeat academic MVP. It uses the SRS as the planned requirements baseline and the project README as the implemented system baseline.

1. Executive Summary

AuraBeat is a full-stack AI music generation platform that allows registered users to describe a track in natural language, select a musical style, and receive an AI-generated MP3 in the browser. The project demonstrates end-to-end web engineering through authentication, protected routes, database-backed profiles and tracks, cloud storage, state-managed audio playback, and integration with an external GPU inference service.

The SRS originally described a large-scale SaaS platform with microservices, payments, API platform capabilities, advanced editing, speech generation, social features, and production-grade GPU infrastructure. The implemented final project is a deliberate academic MVP focused on the core user workflow: account access, prompt-based music generation, saved track management, profile display, and polished UI feedback.

2. Project Overview

Project Name	AuraBeat
Purpose	Generate original music tracks from natural-language prompts using an AI model.
Primary Users	Casual creators, students, content creators, and demo users exploring AI-assisted music creation.
Core Workflow	Register or log in, create a prompt, select style/options, generate audio, save and play tracks.
Deployment Target	Vercel frontend with Supabase backend services and a Google Colab Gradio AI runtime.

3. Requirement Analysis

The SRS defines AuraBeat as an AI-powered music creation platform with account management, credits, generation controls, library features, external APIs, subscriptions, and administration. For the final lab MVP, the team prioritized the subset that could be implemented and demonstrated within academic time and infrastructure constraints.

SRS Area	Original Requirement	MVP Coverage
Authentication	Registration, login, sessions, profile management, OAuth options.	Implemented email/password authentication, route protection, reset flow, profile display/editing, and sign out through Supabase Auth.
AI Music Creation	Prompt/lyrics input, style control, generation output, playback, downloads,	Implemented prompt input, style tag selection, instrumental/vocal option,

	multiple formats.	prompt optimization, Gradio MusicGen integration, MP3 storage, and playback.
Credits	Gold currency, plan tiers, top-ups, refunds, subscription-based pricing.	Implemented Gold balance display and 10 Gold deduction per successful track generation. Plan cards are display-only.
Library	Personal track management, search, playlists, delete, metadata.	Implemented saved track library with search, grid/list views, play action, and delete confirmation.
Admin/API/Community	Admin dashboard, public API, sharing, notifications, referral and affiliate flows.	Mostly descoped for MVP; documented as future work.

4. Implemented Features

4.1 Authentication And Profile

- Email/password registration with display name.
- Login with remember-me behavior and protected application routes.
- Forgot password and reset password workflow with password visibility and strength feedback.
- Profile page with editable display name, current plan display, Gold balance, and sign out.

4.2 Music Generation

- Create page with natural-language prompt input, 500-character limit, style selection, and instrumental/vocal toggle.
- Custom prompt engineering layer that transforms raw user text into a structured MusicGen prompt.
- Server-side integration with a Google Colab T4 GPU Gradio server running Meta MusicGen-small.
- Generated MP3 upload to Supabase Storage with track metadata saved in Supabase PostgreSQL.
- Automatic Gold deduction after successful generation and automatic playback through the global audio player.

4.3 Library And User Experience

- Server-rendered saved track list with client-side search by title, prompt, and tags.
- Grid/list toggle, delete confirmation, and play action integrated with the persistent player.
- Global bottom audio player powered by Zustand so playback persists across pages.
- Toast notification system for success, error, and user-action feedback.
- Dark professional interface with fixed sidebar navigation and purple accent treatment.

5. Architecture And Request Flow

The implemented architecture is a pragmatic monolithic web application rather than the full microservices architecture proposed in the SRS. This decision reduces deployment complexity while still demonstrating the most important engineering behavior: secure user access, API-mediated generation, cloud storage, and persistent playback.

Layer	Implemented Technology	Responsibility
Client/UI	Next.js 14 App Router, React, TypeScript, Tailwind CSS	Pages, forms, sidebar navigation, library, profile, player UI, and responsive states.
State	Zustand	Persistent audio player state and toast notification state.
Auth/Data	Supabase Auth, PostgreSQL, Row Level Security	User identity, profiles, tracks, Gold balance, and protected data access.
Storage	Supabase Storage tracks bucket	Stores generated MP3 files and exposes playable public URLs.
AI Backend	Google Colab T4, Gradio 5.x, Meta MusicGen-small	Runs text-to-music inference and returns generated audio files.
Deployment	Vercel	Hosts the Next.js web application and API routes.

5.1 Generation Request Flow

1. The user submits a prompt and style options on the Create page.
2. The server validates the session and checks that the user has enough Gold.
3. The prompt engineering layer expands the raw prompt into a structured MusicGen-ready prompt.
4. The API route sends the optimized prompt to the Gradio public URL from Google Colab.
5. The generated audio is downloaded and uploaded to Supabase Storage as an MP3.
6. The track row is inserted into the database and 10 Gold are deducted from the profile.
7. The new track is loaded into the global audio player for immediate playback.

6. Database, Storage, And Security

Component	Main Fields / Behavior	Purpose
profiles	id, display_name, gold_balance, plan, created_at, is_admin	Stores user profile data, plan display, Gold balance, and admin flag.
tracks	id, user_id, title, prompt, style_tags, audio_url, duration_seconds, status, created_at	Stores generated track metadata and links each track to its owner.
api_keys	id, user_id, name, key_hash, prefix, is_active	Prepared for API platform use; managed

		with owner-based access policy.
tracks bucket	Public audio storage	Stores generated MP3 files used by playback and library pages.
RLS policies	auth.uid() based access	Prevents users from reading or mutating another user's data.

SECURITY AND STORAGE NOTE: A known production issue remains: deleting a track removes the database row but does not yet remove the MP3 object from Supabase Storage. A production version should store object paths and run a cleanup function.

7. AI Contribution And Academic Honesty

AuraBeat uses Meta MusicGen-small as the pre-trained text-to-music model. The model was not trained from scratch and no custom fine-tuning was performed because free academic infrastructure does not provide enough stable GPU time for model training.

The original AI contribution is the application layer around the model: prompt engineering, Colab/Gradio inference integration, and the complete user-facing workflow from prompt to saved playable audio.

- Prompt Engineering Layer: transforms user descriptions into structured prompts with style, instrumentation, tempo, and texture hints.
- Cloud Inference Integration: connects the Next.js API route to a Gradio server running on Google Colab with T4 GPU acceleration.
- Full Application Workflow: combines authentication, generation, storage, Gold deduction, metadata persistence, and playback.

8. SRS Vs MVP Implementation

SRS Feature	MVP Status	Reason / Explanation
Kafka event streaming	Not implemented	Unnecessary at MVP scale and incompatible with zero-budget deployment goals.
Microservices architecture	Not implemented	A monolithic Next.js architecture is sufficient for the academic demonstration.
Stripe payments	Display-only	Plans are shown, but real billing is outside the academic MVP scope.
Model fine-tuning	Not implemented	Fine-tuning requires more GPU time and curated training data than available.

OAuth social login	Not implemented	Supabase email/password authentication is enough for the submitted workflow.
Mobile responsive layout	Partial	The UI is desktop-first with basic mobile support.
Speech generation and editing tools	Future work	Core music generation was prioritized first.

9. Testing And Validation

The final project should be evaluated through functional, integration, UI, and deployment checks. The most important validation is whether a user can complete the full path from account creation to generated audio playback.

Test Area	Scenario	Expected Result
Authentication	Register, log in, forgot password, reset password, sign out.	User can enter and leave the protected app correctly.
Generation	Submit prompt with valid Gold balance and active Gradio URL.	Track is generated, uploaded, saved, and loaded into the player.
Credits	Generate one successful track.	Gold balance decreases by 10 only after success.
Library	Search, switch view, play track, delete track.	Library updates correctly and player receives selected track.
Profile	Edit display name and view plan/Gold.	Profile data persists through Supabase.
Failure Handling	Use missing/expired Gradio URL or insufficient Gold.	User receives clear error toast and no broken UI state remains.

10. Deployment And Setup

10.1 Local Setup

8. Clone the AuraBeat repository from GitHub.
9. Install dependencies with npm install.
10. Create a .env.local file with Supabase URL, Supabase anon key, and GRADIO_SERVER_URL.
11. Run npm run dev and open http://localhost:3000.
12. Start the Google Colab MusicGen notebook, copy the Gradio public URL, and update GRADIO_SERVER_URL whenever the Colab session restarts.

10.2 Environment Variables

NEXT_PUBLIC_SUPABASE_URL	Supabase project URL used by the frontend and server helpers.
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anonymous key for authenticated client access.
GRADIO_SERVER_URL	Temporary public Gradio URL for the Colab MusicGen backend.

11. Marking Scheme Alignment

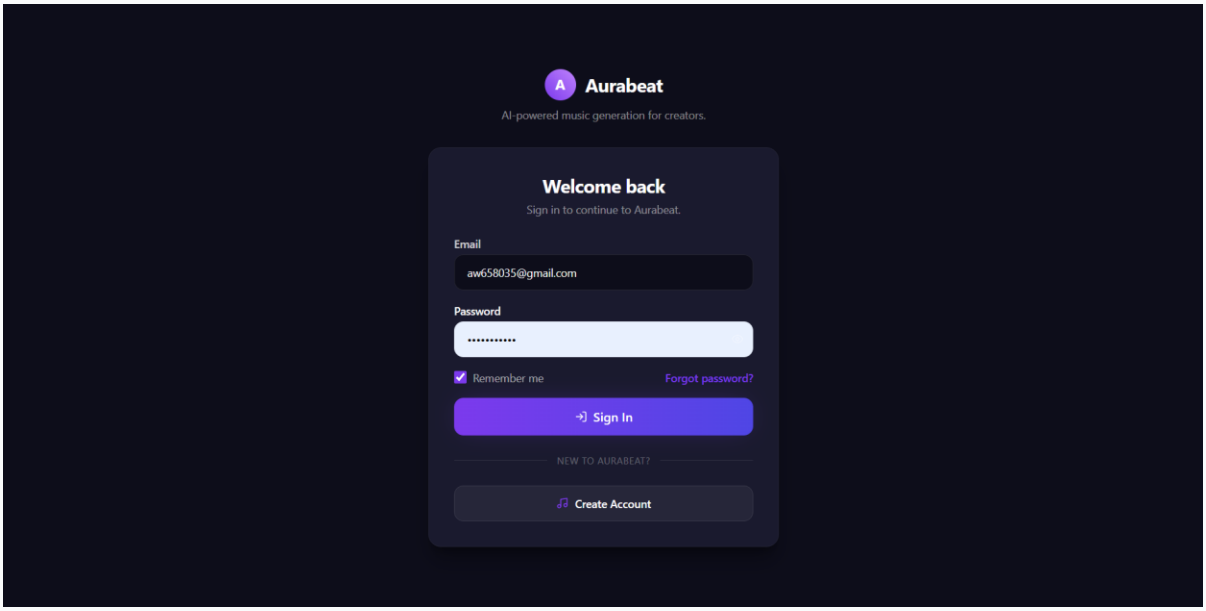
Assessment Component	Evidence In AuraBeat	Status
Project Requirements	SRS-backed AI music generation platform with documented MVP scope.	Completed
Frontend Implementation	Next.js pages, reusable components, protected routes, sidebar, player, library, and profile screens.	Completed
Backend / API	Next.js API route for generation, Supabase integration, Gradio inference pipeline.	Completed
Database / Storage	Supabase profiles, tracks, api_keys, RLS policies, Storage tracks bucket.	Completed
Code Quality / Documentation	README includes setup, architecture, feature list, limitations, viva explanation, and future work.	Completed
Deployment	Vercel deployment path documented; Gradio Colab dependency documented.	Completed with dependency

12. Known Limitations And Future Work

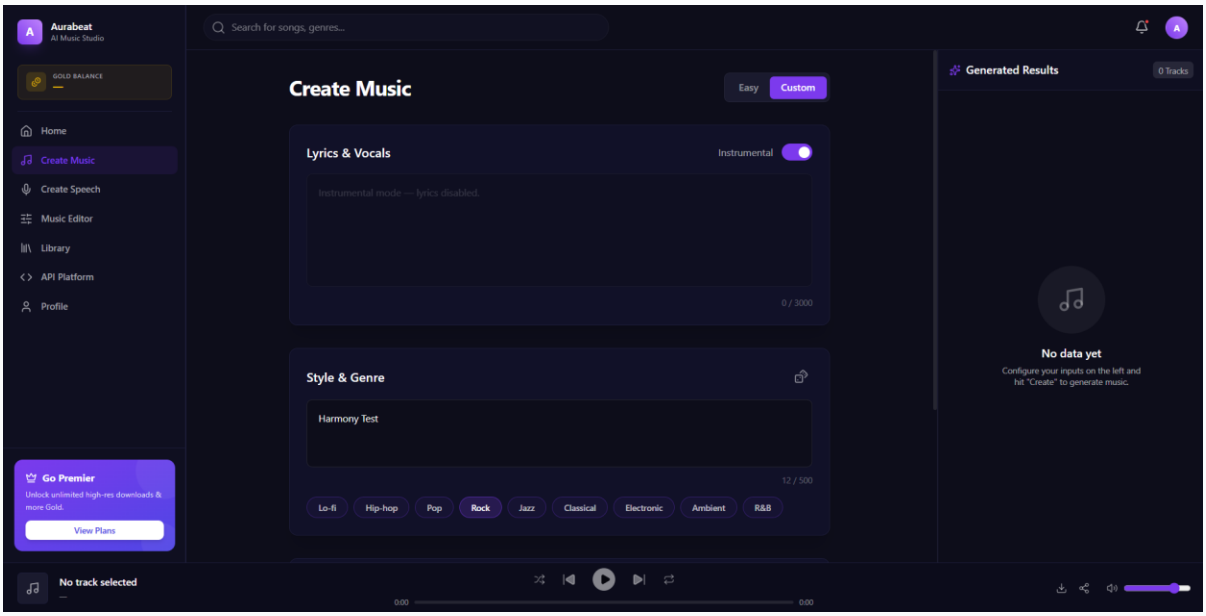
- The Gradio URL is temporary and must be refreshed when the Colab session restarts.
- Google Colab free tier may disconnect after inactivity or GPU usage limits.
- The current /api/generate route should verify identity from the server-side Supabase session in production.
- Deleting a track should also remove the stored MP3 object in a production release.
- Future work includes persistent GPU hosting, Stripe payments, mobile-first layout, waveform visualization, sharing, analytics, batch generation, and model fine-tuning.

Appendix A: Screenshot Placeholders

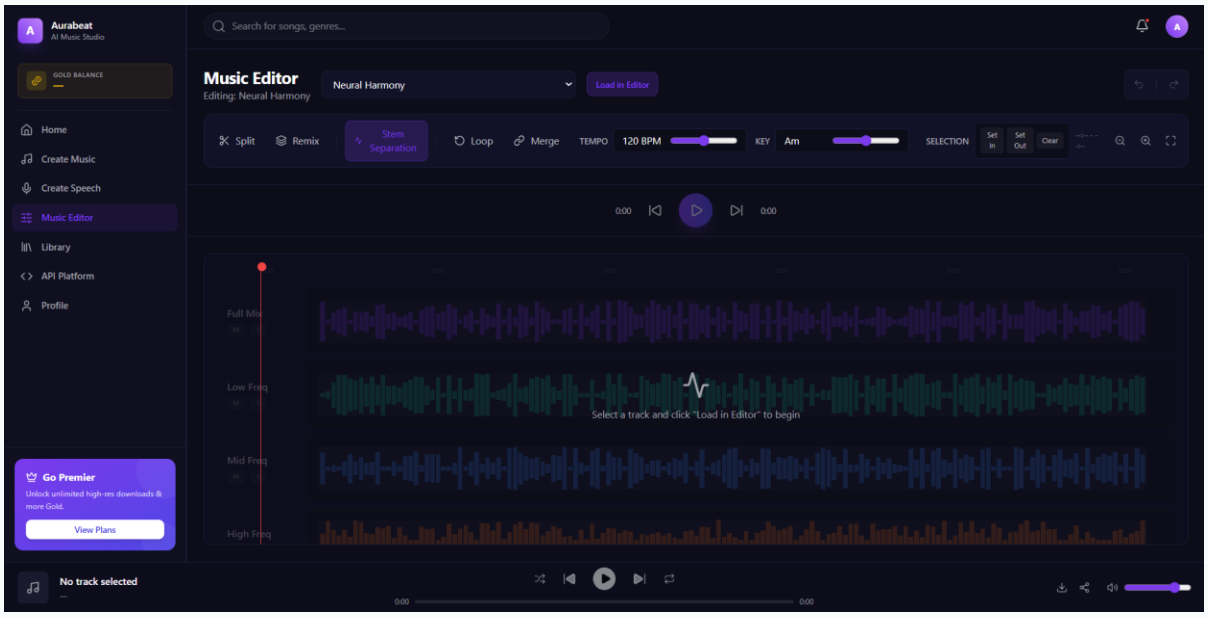
Insert actual project screenshots in the placeholders below before final printed submission if required by the instructor.



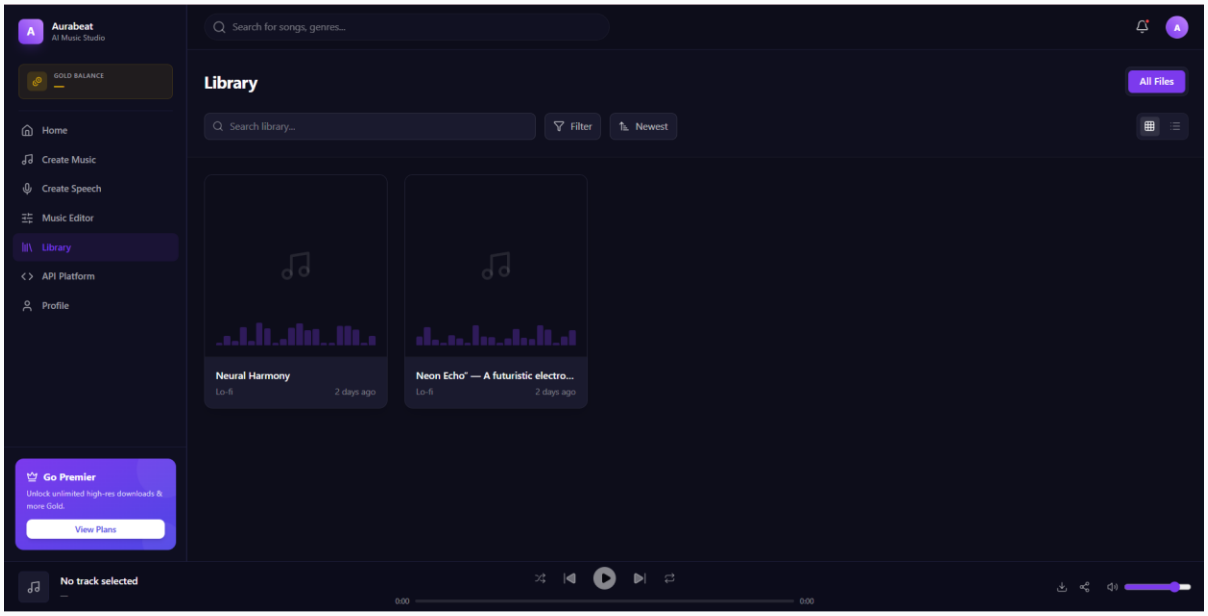
Show email/password authentication and account entry point.



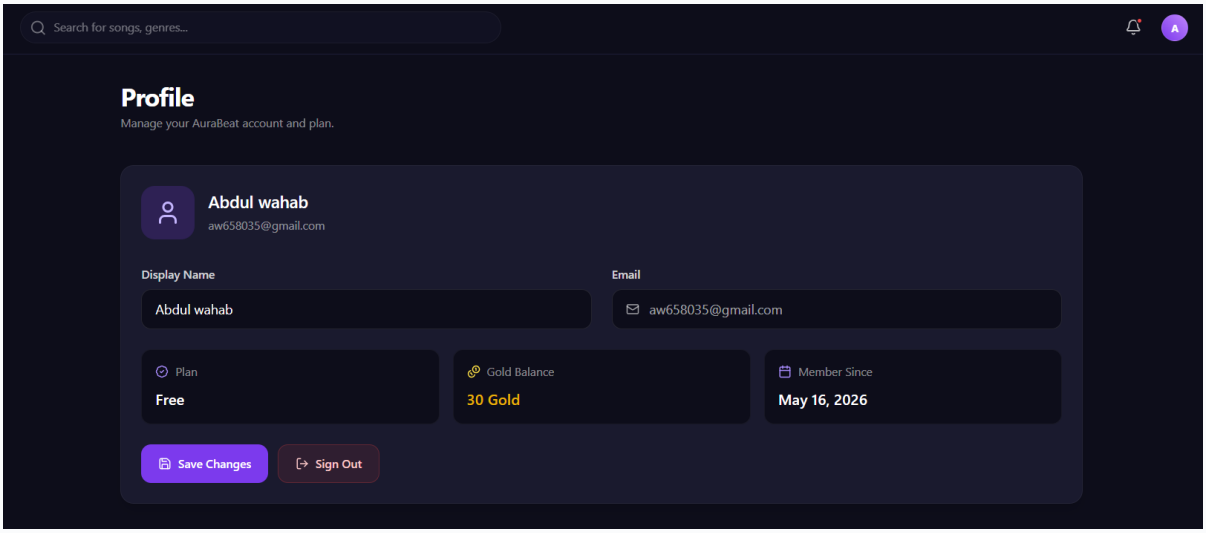
Show prompt input, style selector, instrumental/vocal option, and create action.



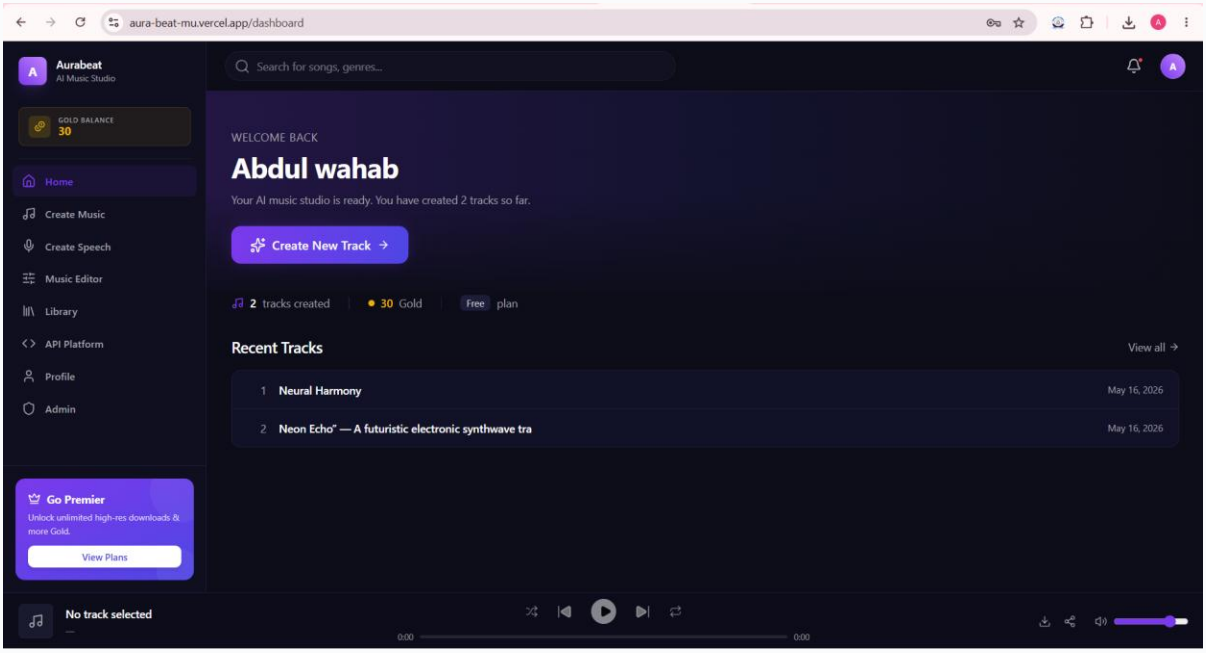
Show generated result and persistent bottom audio player.



Show saved tracks, search, grid/list toggle, play, and delete options.



Show display name editing, plan display, Gold balance, and sign out.



Show deployed Vercel app or local running application.

Appendix B: Submission Signatures

Instructor's Signature	_____ Date: 18/May/2026
Student Name / Roll No.	ABDUL WAHAB (007)
Student Name / Roll No.	TALHA NAVEED (089)